Wstępna analiza funkcjonalna aplikacji

Projekt średni nr 2

Błędnik

Symulator ruchu drogowego

Bartosz Młynarski

---

1. Cel i ogólna koncepcja programu

Celem projektu jest stworzenie aplikacji symulującej ruch pojazdów w uproszczonym środowisku miejskim. Program będzie modelował poruszanie się samochodów po sieci ulic z uwzględnieniem podstawowych zasad ruchu drogowego, sygnalizacji świetlnej oraz obecności pieszych.

Symulacja będzie realizowana w trybie krokowym (tick-based). Oznacza to, że czas w systemie będzie podzielony na dyskretne kroki, a w każdym kroku aktualizowany będzie stan całej symulacji. Takie podejście upraszcza logikę oraz pozwala w sposób deterministyczny wykrywać kolizje i sytuacje niebezpieczne.

Aplikacja będzie posiadać interfejs graficzny wykonany w technologii Qt, umożliwiający obserwację ruchu pojazdów oraz kontrolę przebiegu symulacji.

2. Model działania symulacji

Mapa zostanie zaimplementowana jako graf skierowany:

- Węzły (Node) – reprezentują skrzyżowania.
- Krawędzie (Edge) – reprezentują odcinki dróg między skrzyżowaniami.

Każda krawędź zostanie podzielona na określoną liczbę segmentów. Segment jest najmniejszą jednostką przestrzeni, jaką może zajmować pojazd. W jednym segmencie może znajdować się maksymalnie jeden pojazd.

Każdy krok symulacji będzie przebiegał według ustalonego schematu:

1. Aktualizacja stanu sygnalizacji świetlnej (zmiana fazy w oparciu o timer).
2. Losowe generowanie pieszych na przejściach.
3. Faza decyzyjna pojazdów:
    - sprawdzenie stanu światła,
    - sprawdzenie dostępności kolejnego segmentu,
    - podjęcie decyzji o ruchu.
4. Aktualizacja pozycji pojazdów.
5. Wykrycie kolizji i sytuacji niebezpiecznych.
6. Rejestracja zdarzeń w logu.

3. Sygnalizacja świetlna

Na wybranych skrzyżowaniach zostanie zaimplementowana sygnalizacja świetlna. Każda sygnalizacja będzie posiadać:

- stan (RED / GREEN),
- licznik czasu do zmiany fazy.

4. Piesi

Piesi będą modelowani w uproszczony sposób:

- Losowo generowane zdarzenie „zajęte przejście”.
- Przez określoną liczbę kroków przejście będzie blokowane.
- Pojazdy przed przejściem będą zobowiązane do zatrzymania.

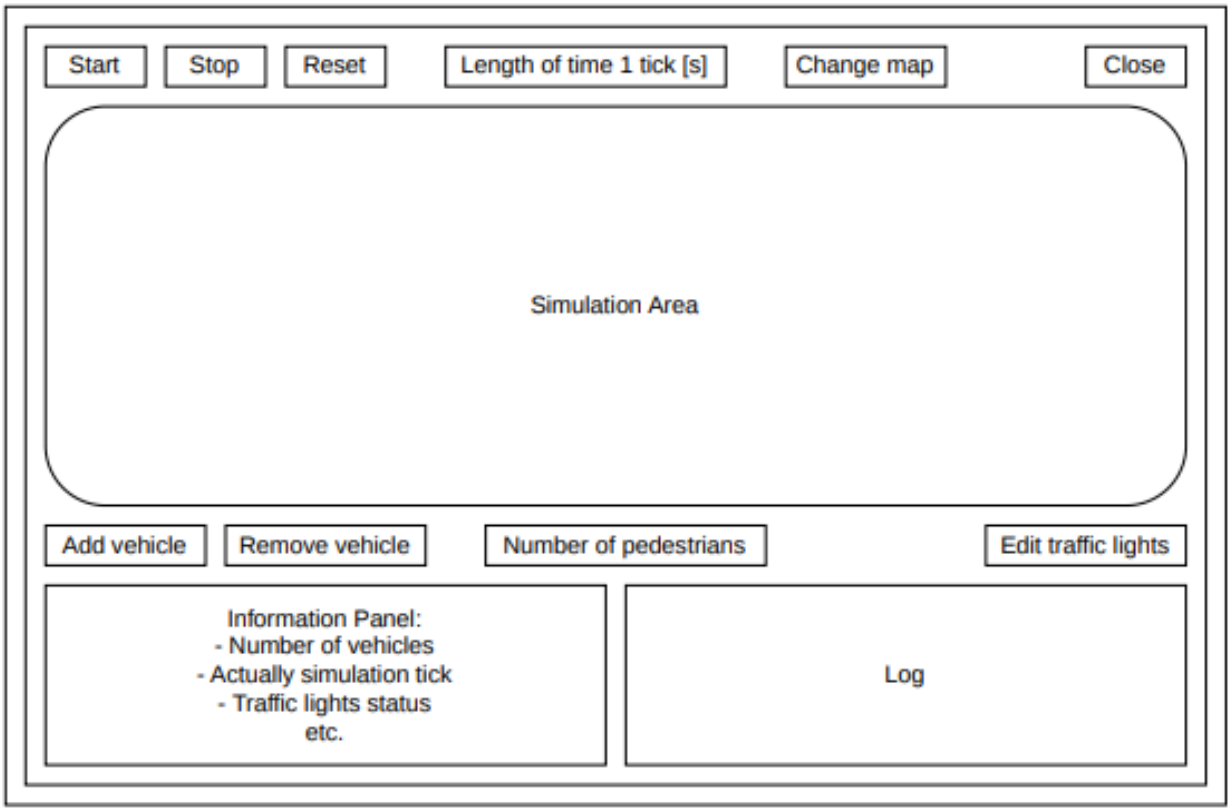
Nie będzie symulowany fizyczny ruch pieszego w przestrzeni 2D – celem jest uproszczenie modelu przy zachowaniu wpływu na ruch drogowy.

5. Wykrywanie kolizji i sytuacji niebezpiecznych

System będzie wykrywał:

1. Kolizję – gdy dwa pojazdy próbują zająć ten sam segment w tym samym kroku.
2. Przejazd na czerwonym świetle.
3. Wymuszenie pierwszeństwa.
4. Zator drogowy (np. brak możliwości ruchu przez określoną liczbę kroków).

6. Interfejs użytkownika



W obszarze graficznym będą widoczne:

- drogi jako linie,
- skrzyżowania jako punkty,
- pojazdy jako prostokąty,
- sygnalizacja jako kolorowe wskaźniki (czerwony/zielony).

7. Środowisko programistyczne

- Język: C++
- Środowisko: Qt Creator
- Kompilator: MinGW 64-bit (Windows)

Wykorzystane biblioteki:

- Standard Template Library (vector, queue, unordered_map, algorithm)
- Qt Core / Gui / Widgets do interfejsu graficznego

Nie są planowane zewnętrzne biblioteki dodatkowe.

8. Architektura programu

GUI / Prezentacja _____

MainWindow - Główne okno aplikacji. Obsługuje interfejs użytkownika, przyciski sterujące symulacją, timer oraz komunikację z warstwą logiki.

MapView - Widok mapy oparty o Qt Graphics View. Odpowiada za rysowanie dróg, pojazdów, świateł oraz aktualizację elementów na ekranie.

VehicleItem - Graficzna reprezentacja pojedynczego pojazdu na mapie. Śledzi obiekt Vehicle i aktualizuje jego pozycję wizualną.

Logika / Model _____

Simulation - Główna klasa zarządzająca całą symulacją. Przechowuje mapę, pojazdy, światła, zdarzenia drogowe i wykonuje kolejne ticki symulacji.

Graph - Model sieci drogowej w postaci grafu skierowanego. Zawiera węzły i drogi oraz umożliwia wyszukiwanie tras.

Node - Punkt mapy lub skrzyżowanie. Przechowuje współrzędne, połączenia drogowe oraz informacje o przejściach dla pieszych.

Edge - Odcinek drogi łączący dwa węzły. Zawiera informacje o zajętości pasa oraz obecności pojazdu uprzywilejowanego.

TrafficLight - Model sygnalizacji świetlnej przypisanej do skrzyżowania. Steruje ruchem i może przejść w tryb awaryjny.

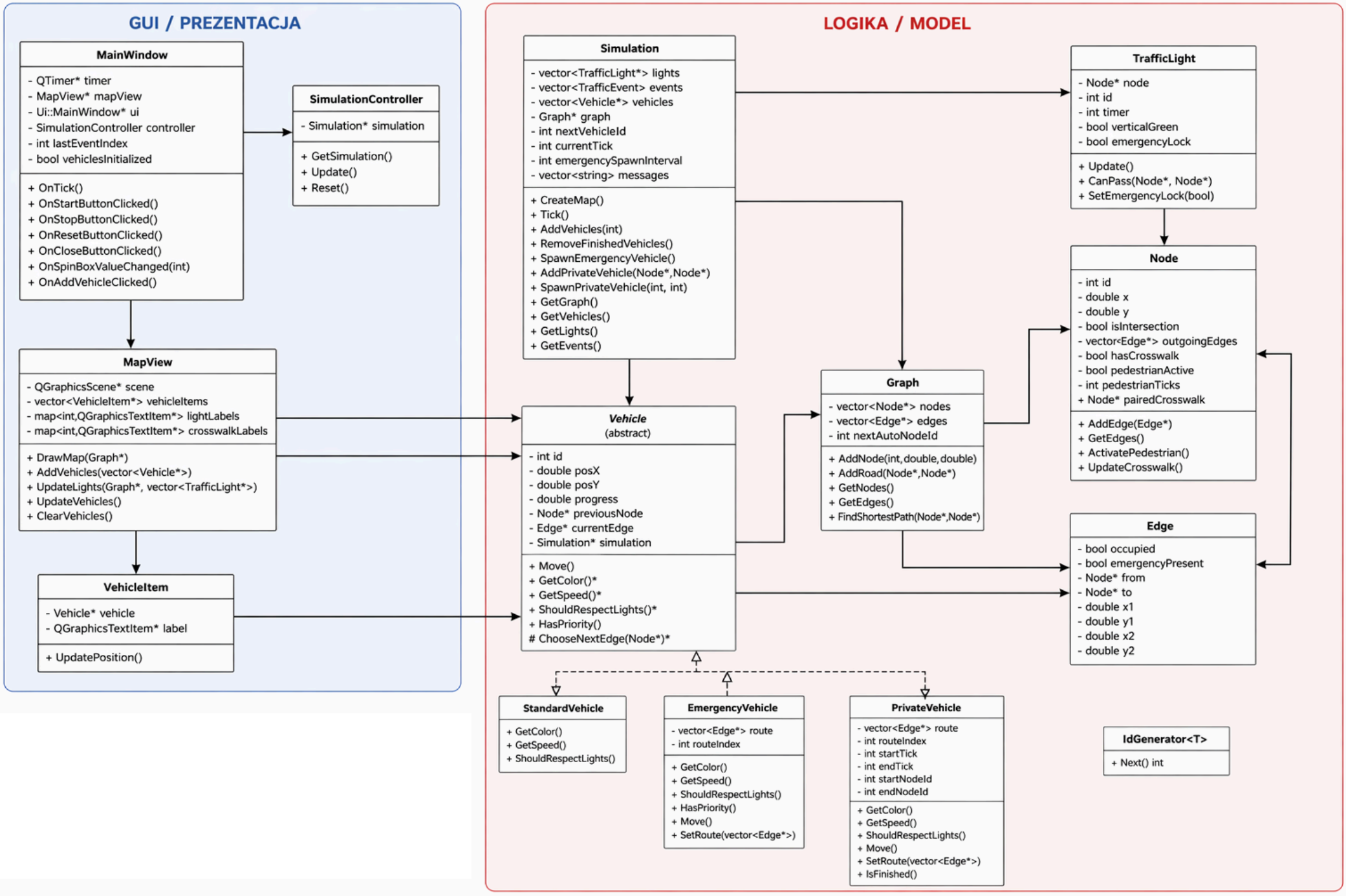
Vehicle - Abstrakcyjna klasa bazowa wszystkich pojazdów. Definiuje wspólne dane, pozycję, ruch oraz interfejs dla klas dziedziczących.

StandardVehicle - Zwykły pojazd poruszający się zgodnie z zasadami ruchu. Respektuje światła i wybiera kolejne drogi automatycznie.

EmergencyVehicle - Pojazd uprzywilejowany. Ma pierwszeństwo przejazdu, może ignorować sygnalizację i korzysta z wyznaczonej trasy.

PrivateVehicle - Pojazd prywatny jadący z konkretnego punktu startowego do celu. Porusza się po wcześniej wyliczonej trasie i mierzy czas przejazdu.

IdGenerator<T> - Szablonowa klasa generująca unikalne identyfikatory dla różnych typów obiektów w systemie.


GUI / PREZENTACJA
MainWindow
- QTimer* timer
- MapView* mapView
- Ui::MainWindow* ui
- SimulationController controller
- int lastEventIndex
- bool vehiclesInitialized
+ OnTick()
+ OnStartButtonClicked()
+ OnStopButtonClicked()
+ OnResetButtonClicked()
+ OnCloseButtonClicked()
+ OnSpinBoxValueChanged(int)
+ OnAddVehicleClicked()
SimulationController
- Simulation* simulation
+ GetSimulation()
+ Update()
+ Reset()
MapView
- QGraphicsScene* scene
- vector<VehicleItem*> vehicleItems
- map<int,QGraphicsTextItem*> lightLabels
- map<int,QGraphicsTextItem*> crosswalkLabels
+ DrawMap(Graph*)
+ AddVehicles(vector<Vehicle*>)
+ UpdateLights(Graph*, vector<TrafficLight*>)
+ UpdateVehicles()
+ ClearVehicles()
VehicleItem
- Vehicle* vehicle
- QGraphicsTextItem* label
+ UpdatePosition()
LOGIKA / MODEL
Simulation
- vector<TrafficLight*> lights
- vector<TrafficEvent> events
- vector<Vehicle*> vehicles
- Graph* graph
- int nextVehicleId
- int currentTick
- int emergencySpawnInterval
- vector<string> messages
+ CreateMap()
+ Tick()
+ AddVehicles(int)
+ RemoveFinishedVehicles()
+ SpawnEmergencyVehicle()
+ AddPrivateVehicle(Node*,Node*)
+ SpawnPrivateVehicle(int, int)
+ GetGraph()
+ GetVehicles()
+ GetLights()
+ GetEvents()
TrafficLight
- Node* node
- int id
- int timer
- bool verticalGreen
- bool emergencyLock
+ Update()
+ CanPass(Node*, Node*)
+ SetEmergencyLock(bool)
Node
- int id
- double x
- double y
- bool isIntersection
- vector<Edge*> outgoingEdges
- bool hasCrosswalk
- bool pedestrianActive
- int pedestrianTicks
+ Node* pairedCrosswalk
+ AddEdge(Edge*)
+ GetEdges()
+ ActivatePedestrian()
+ UpdateCrosswalk()
Graph
- vector<Node*> nodes
- vector<Edge*> edges
- int nextAutoNodeId
+ AddNode(int,double,double)
+ AddRoad(Node*,Node*)
+ GetNodes()
+ GetEdges()
+ FindShortestPath(Node*,Node*)
Vehicle
(abstract)
- int id
- double posX
- double posY
- double progress
- Node* previousNode
- Edge* currentEdge
- Simulation* simulation
+ Move()
+ GetColor()*
+ GetSpeed()*
+ ShouldRespectLights()*
+ HasPriority()
# ChooseNextEdge(Node*)*
Edge
- bool occupied
- bool emergencyPresent
- Node* from
- Node* to
- double x1
- double y1
- double x2
- double y2
StandardVehicle
+ GetColor()
+ GetSpeed()
+ ShouldRespectLights()
EmergencyVehicle
- vector<Edge*> route
- int routeIndex
+ GetColor()
+ GetSpeed()
+ ShouldRespectLights()
+ HasPriority()
+ Move()
+ SetRoute(vector<Edge*>)
PrivateVehicle
- vector<Edge*> route
- int routeIndex
- int startTick
- int endTick
- int startNodeId
- int endNodeId
+ GetColor()
+ GetSpeed()
+ ShouldRespectLights()
+ Move()
+ SetRoute(vector<Edge*>)
+ IsFinished()
IdGenerator<T>
+ Next() int

9. Zrealizowane funkcjonalności

W trakcie realizacji projektu wykonano następujące elementy:

- utworzenie sieci dróg opartej na strukturze grafu,
- implementacja węzłów oraz połączeń drogowych,
- symulacja ruchu pojazdów,
- obsługa różnych typów pojazdów:
    - pojazdy standardowe,
    - pojazdy prywatne poruszające się po wyznaczonej trasie,
    - pojazdy uprzywilejowane posiadające pierwszeństwo,
- system sygnalizacji świetlnej na skrzyżowaniach,
- przejścia dla pieszych,
- wyznaczanie tras z wykorzystaniem najkrótszej ścieżki,
- rejestrowanie zdarzeń i komunikatów z symulacji,
- graficzna wizualizacja działania programu,
- interfejs użytkownika umożliwiający sterowanie symulacją,
- regulacja szybkości działania programu.

10. Napotkane problemy

Podczas realizacji projektu największymi wyzwaniami były:

- synchronizacja ruchu wielu pojazdów jednocześnie,
- logika przejazdu przez skrzyżowania,
- obsługa pierwszeństwa pojazdów uprzywilejowanych,
- wyznaczanie tras pomiędzy punktami mapy,
- połączenie logiki programu z interfejsem graficznym.

11. Efekt końcowy

Końcowa wersja programu poprawnie symuluje ruch drogowy na mapie miasta, umożliwia interakcję użytkownika oraz przedstawia stan pojazdów i skrzyżowań w czasie rzeczywistym.

Projekt spełnia założenia funkcjonalne oraz wymagania techniczne.

Repozytorium GIT: https://github.com/M1yn4rz/Road-Traffic-Simulator